Binary Tree Data Structure

What is a Binary Tree?

A binary tree is a hierarchical data structure where each node has at most two children, referred to as the left child and the right child. The topmost node is called the root, and nodes with no children are called leaves.

Binary Tree Node Structure

```java
class TreeNode {
    int data;
    TreeNode left;
    TreeNode right;

    // Constructor
    public TreeNode(int data) {
        this.data = data;
        this.left = null;
        this.right = null;
    }
}
```

Types of Binary Trees

Complete Binary Tree

A binary tree where all levels are completely filled except possibly the last level, which is filled from left to right.

Full Binary Tree

A binary tree where every node has either 0 or 2 children.

Perfect Binary Tree

A binary tree where all internal nodes have two children and all leaves are at the same level.

Binary Search Tree (BST)

A binary tree where for each node, all values in the left subtree are less than the node's value, and all values in the right subtree are greater than the node's value.

Basic Binary Tree Operations

Creating a Binary Tree

```java
public class BinaryTree {
    TreeNode root;

    public BinaryTree() {
        root = null;
    }

    // Create a simple binary tree
    public void createTree() {
        root = new TreeNode(1);
        root.left = new TreeNode(2);
        root.right = new TreeNode(3);
        root.left.left = new TreeNode(4);
        root.left.right = new TreeNode(5);
    }
}
```

Tree Traversal Methods

Inorder Traversal (Left, Root, Right)

```java
public void inorderTraversal(TreeNode node) {
    if (node != null) {
        inorderTraversal(node.left);
        System.out.print(node.data + " ");
        inorderTraversal(node.right);
    }
}
```

Preorder Traversal (Root, Left, Right)

```java
public void preorderTraversal(TreeNode node) {
    if (node != null) {
        System.out.print(node.data + " ");
        preorderTraversal(node.left);
        preorderTraversal(node.right);
    }
}
```

Postorder Traversal (Left, Right, Root)

```java
public void postorderTraversal(TreeNode node) {
    if (node != null) {
        postorderTraversal(node.left);
        postorderTraversal(node.right);
        System.out.print(node.data + " ");
    }
}
```

Level Order Traversal (Breadth-First)

```java
import java.util.Queue;
import java.util.LinkedList;

public void levelOrderTraversal(TreeNode root) {
    if (root == null) return;

    Queue<TreeNode> queue = new LinkedList<>();
    queue.offer(root);

    while (!queue.isEmpty()) {
        TreeNode current = queue.poll();
        System.out.print(current.data + " ");

        if (current.left != null) {
            queue.offer(current.left);
        }
        if (current.right != null) {
            queue.offer(current.right);
        }
    }
}
```

Common Binary Tree Operations

Finding Height of Binary Tree

```java
public int height(TreeNode node) {
    if (node == null) {
        return -1; // or 0 depending on definition
    }

    int leftHeight = height(node.left);
    int rightHeight = height(node.right);

    return Math.max(leftHeight, rightHeight) + 1;
}
```

Counting Nodes in Binary Tree

```java
public int countNodes(TreeNode node) {
    if (node == null) {
        return 0;
    }

    return 1 + countNodes(node.left) + countNodes(node.right);
}
```

Finding Maximum Value in Binary Tree

```java
public int findMax(TreeNode node) {
    if (node == null) {
        return Integer.MIN_VALUE;
    }

    int leftMax = findMax(node.left);
    int rightMax = findMax(node.right);

    return Math.max(node.data, Math.max(leftMax, rightMax));
}
```

Searching for a Value

```java
public boolean search(TreeNode node, int value) {
    if (node == null) {
        return false;
    }

    if (node.data == value) {
        return true;
    }

    return search(node.left, value) || search(node.right, value);
}
```

Binary Search Tree Operations

Insertion in BST

```java
public TreeNode insertBST(TreeNode root, int value) {
    if (root == null) {
        return new TreeNode(value);
    }

    if (value < root.data) {
        root.left = insertBST(root.left, value);
    } else if (value > root.data) {
        root.right = insertBST(root.right, value);
    }

    return root;
}
```

Search in BST

```java
public boolean searchBST(TreeNode root, int value) {
    if (root == null) {
        return false;
    }

    if (root.data == value) {
        return true;
    }

    if (value < root.data) {
        return searchBST(root.left, value);
    } else {
        return searchBST(root.right, value);
    }
}
```

Delete from BST

```java
public TreeNode deleteBST(TreeNode root, int value) {
    if (root == null) {
        return null;
    }

    if (value < root.data) {
        root.left = deleteBST(root.left, value);
    } else if (value > root.data) {
        root.right = deleteBST(root.right, value);
    } else {
        // Node to be deleted found
        if (root.left == null) {
            return root.right;
        } else if (root.right == null) {
            return root.left;
        }

        // Node with two children
        root.data = findMinValue(root.right);
        root.right = deleteBST(root.right, root.data);
    }

    return root;
}

private int findMinValue(TreeNode node) {
    int minValue = node.data;
    while (node.left != null) {
        minValue = node.left.data;
        node = node.left;
    }
    return minValue;
}
```

Practice Session On Binary Tree

What is the time complexity of searching for an element in a balanced binary search tree?

Given a binary tree, write a method to check if it is a valid binary search tree.

How would you find the lowest common ancestor of two nodes in a binary tree?

What is the difference between a complete binary tree and a full binary tree?

How would you serialize and deserialize a binary tree?

What is the time complexity of inserting an element in a binary search tree in the worst case?

How would you find the diameter of a binary tree (longest path between any two nodes)?

What is the space complexity of the recursive inorder traversal?

How would you convert a sorted array into a balanced binary search tree?

What is the difference between depth-first search and breadth-first search in binary trees?

How would you find all paths from root to leaf in a binary tree?

What is the time complexity of finding the height of a binary tree?

How would you check if two binary trees are identical?

What is the difference between a binary tree and a binary search tree?

How would you find the kth smallest element in a binary search tree?

Quiz Questions

1. What is the maximum number of nodes at level 'h' in a binary tree? a) 2^h b) 2^(h-1) c) 2^(h+1) d) h^2

Answer: a) 2^h

2. In which traversal method do we visit the root node first? a) Inorder b) Postorder c) Preorder d) Level order

Answer: c) Preorder

3. What is the time complexity of searching in a balanced binary search tree? a) O(1) b) O(log n) c) O(n) d) O(n log n)

Answer: b) O(log n)

4. In a complete binary tree with n nodes, what is the height? a) $\log_2(n)$ b) $\lfloor \log_2(n) \rfloor$ c) $\lceil \log_2(n+1) \rceil - 1$ d) n/2

Answer: c) $\lceil \log_2(n+1) \rceil - 1$

5. Which data structure is typically used for level order traversal? a) Stack b) Queue c) Array d) Linked List

Answer: b) Queue

Practice Coding Questions

1. Write a Java method to find the maximum depth (height) of a binary tree.

2. Implement a method to check if a binary tree is symmetric (mirror image of itself).

3. Write a function to find the sum of all nodes in a binary tree.

4. Create a method to print all leaf nodes of a binary tree.

5. Implement a function to find the level of a given node in a binary tree.